C++ Chapter 05 Notes

Chapter 05 — Notes

5.1 — Constant variables (named constants)

Introduction to constants

In programming, a **constant** is a value that may not be changed during the program's execution.

C++ supports two different kinds of constants:

- **Named constants** are constant values that are associated with an identifier. These are also sometimes called **symbolic constants**.
- **Literal constants** are constant values that are not associated with an identifier.

We'll start our coverage of constants by looking at named constants. We will then cover literal constants (in upcoming lesson 5.2 – Literals).

Types of named constants

There are three ways to define a named constant in C++:

- Constant variables (covered in this lesson).
- Object-like macros with substitution text (introduced in lesson 2.10 – Introduction to the preprocessor, with additional coverage in this lesson).
- Enumerated constants (covered in lesson 13.2 – Unscoped enumerations).

Constant variables are the most common type of named constant, so we'll start there.

Constant variables

So far, all of the variables we've seen have been non-constant – that is, their values can be changed at any time (typically by assigning a new value). For example:

```
int main()
{
    int x { 4 }; // x is a non-constant variable
    x = 5; // change value of x to 5 using assignment operator

    return 0;
}
```

However, there are many cases where it is useful to define variables with values that can not be changed. For example, consider the gravity of Earth (near the surface): 9.8 meters/second2. This isn't likely to change any time soon (and if it does, you've likely got bigger problems than learning C++). Defining this value as a constant helps ensure that this value isn't accidentally changed. Constants also have other benefits that we'll explore in subsequent lessons.

Although it is a well-known oxymoron, a variable whose value cannot be changed after initialization is called a **constant variable**.

Declaring a const variable

To declare a constant variable, we place the `const` keyword (called a "const qualifier") adjacent to the object's type:

```
const double gravity { 9.8 };  // preferred use of const before type
int const sidesInSquare { 4 }; // "east const" style, okay but not preferred
```

Although C++ will accept the const qualifier either before or after the type, it's much more common to use `const` before the type because it better follows standard English language convention where modifiers come before the object being modified (e.g. a "a green ball", not a "a ball green").

> **As an aside…**
>
> Due to the way that the compiler parses more complex declarations, some developers prefer placing the `const` after the type (because it is slightly more consistent). This style is called "east const". While this style has some advocates (and some reasonable points), it has not caught on significantly.

Best practice

Place `const` before the type (because it is more conventional to do so).

Key insight

The type of an object includes the const qualifier, so when we define `const double gravity { 9.8 };` the type of `gravity` is `const double`.

Const variables must be initialized

Const variables *must* be initialized when you define them, and then that value can not be changed via assignment:

```
int main()
{
    const double gravity; // error: const variables must be initialized
    gravity = 9.9;        // error: const variables can not be changed

    return 0;
}
```

Note that const variables can be initialized from other variables (including non-const ones):

```
#include <iostream>

int main()
{
    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    const int constAge { age }; // initialize const variable using non-const value

    age = 5;      // ok: age is non-const, so we can change its value
    constAge = 6; // error: constAge is const, so we cannot change its value

    return 0;
}
```

In the above example, we initialize const variable `constAge` with non-const variable `age`. Because `age` is still non-const, we can change its value. However, because `constAge` is const, we cannot change the value it has after initialization.

Key insight

The initializer of a const variable can be a non-constant value.

Naming your const variables

There are a number of different naming conventions that are used for const variables.

Programmers who have transitioned from C often prefer underscored, upper-case names for const variables (e.g. `EARTH_GRAVITY`). More common in C++ is to use intercapped names with a 'k' prefix (e.g. `kEarthGravity`).

However, because const variables act like normal variables (except they can not be assigned to), there is no reason that they need a special naming

convention. For this reason, we prefer using the same naming convention that we use for non-const variables (e.g. `earthGravity`).

Const function parameters

Function parameters can be made constants via the `const` keyword:

```
#include <iostream>

void printInt(const int x)
{
    std::cout << x << '\n';
}

int main()
{
    printInt(5); // 5 will be used as the initializer for x
    printInt(6); // 6 will be used as the initializer for x

    return 0;
}
```

Note that we did not provide an explicit initializer for our const parameter `x` – the value of the argument in the function call will be used as the initializer for `x`.

Making a function parameter constant enlists the compiler's help to ensure that the parameter's value is not changed inside the function. However, in modern C++ we don't make value parameters `const` because we generally don't care if the function changes the value of the parameter (since it's just a copy that will be destroyed at the end of the function anyway). The `const` keyword also adds a small amount of unnecessary clutter to the function prototype.

> **Best practice**
>
> Don't use `const` for value parameters.

Later in this tutorial series, we'll talk about two other ways to pass arguments to functions: pass by reference, and pass by address. When using either of these methods, proper use of `const` is important.

Const return values

A function's return value may also be made const:

```
#include <iostream>

const int getValue()
{
    return 5;
}

int main()
{
    std::cout << getValue() << '\n';

    return 0;
}
```

For fundamental types, the `const` qualifier on a return type is simply ignored (your compiler may generate a warning).

For other types (which we'll cover later), there is typically little point in returning const objects by value, because they are temporary copies that will be destroyed anyway. Returning a const value can also impede certain kinds of compiler optimizations (involving move semantics), which can result in lower performance.

> **Best practice**
>
> Don't use `const` when returning by value.

Why variables should be made constant

If a variable can be made constant, it generally should be made constant. This is important for a number of reasons:

- It reduces the chance of bugs. By making a variable constant, you ensure that the value can't be changed accidentally.
- It provides more opportunity for the compiler to optimize programs. When the compiler can assume a value isn't changing, it is able to leverage more techniques to optimize the program, resulting in a compiled program that is smaller and faster. We'll discuss this further later in this chapter.
- Most importantly, it reduces the overall complexity of our programs. When trying to determine what a section of code is doing or trying to debug an issue, we know that a const variable can't have its value changed, so we don't have to worry about whether its value is actually changing, what value it is changing to, and whether that new value is correct.

> **Key insight**
>
> Every moving part in a system increases complexity and the risk of defect or failure. Non-constant variables are moving parts, while constant variables are not.

> **Best practice**
>
> Make variables constant whenever possible. Exception cases include by-value function parameters and by-value return types, which should generally not be made constant.

Object-like macros with substitution text

In lesson 2.10 – Introduction to the preprocessor, we discussed object-like macros with substitution text. For example:

```
#include <iostream>

#define MY_NAME "Alex"

int main()
{
    std::cout << "My name is: " << MY_NAME << '\n';

    return 0;
}
```

When the preprocessor processes the file containing this code, it will replace `MY_NAME` (on line 7) with `"Alex"`. Note that `MY_NAME` is a name, and the substitution text is a constant value, so object-like macros with substitution text are also named constants.

Prefer constant variables to preprocessor macros

So why not use preprocessor macros for named constants? There are (at least) three major problems.

The biggest issue is that macros don't follow normal C++ scoping rules. Once a macro is #defined, all subsequent occurrences of the macro's name in the current file will be replaced. If that name is used elsewhere, you'll get macro substitution where you didn't want it. This will most likely lead to strange

compilation errors. For example:

```
#include <iostream>

void someFcn()
{
// Even though gravity is defined inside this function
// the preprocessor will replace all subsequent occurrences of gravity in the rest of the file
#define gravity 9.8
}

void printGravity(double gravity) // including this one, causing a compilation error
{
    std::cout << "gravity: " << gravity << '\n';
}

int main()
{
    printGravity(3.71);

    return 0;
}
```

When compiled, GCC produced this confusing error:

```
prog.cc:7:17: error: expected ',' or '...' before numeric constant
    5 | #define gravity 9.8
      |                 ^~~
prog.cc:10:26: note: in expansion of macro 'gravity'
```

Second, it is often harder to debug code using macros. Although your source code will have the macro's name, the compiler and debugger never see the macro because it has already been replaced before they run. Many debuggers are unable to inspect a macro's value, and often have limited capabilities when working with macros.

Third, macro substitution behaves differently than everything else in C++. Inadvertent mistakes can be easily made as a result.

Constant variables have none of these problems: they follow normal scoping rules, can be seen by the compiler and debugger, and behave consistently.

> **Best practice**
>
> Prefer constant variables over object-like macros with substitution text.

Using constant variables throughout a multi-file program

In many applications, a given named constant needs to be used throughout your code (not just in one file). These can include physics or mathematical constants that don't change (e.g. pi or Avogadro's number), or application-specific "tuning" values (e.g. friction or gravity coefficients). Instead of redefining these every time they are needed, it's better to declare them once in a central location and use them wherever needed. That way, if you ever need to change them, you only need to change them in one place.

There are multiple ways to facilitate this within C++ – we cover this topic in full detail in lesson 7.10 – Sharing global constants across multiple files (using inline variables).

Nomenclature: type qualifiers

A **type qualifier** (sometimes called a **qualifier** for short) is a keyword that is applied to a type that modifies how that type behaves. The `const` used to

declare a constant variable is called a **const type qualifier** (or **const qualifier** for short).

As of C++23, C++ only has two type qualifiers: `const` and `volatile`.

> **Optional reading**
>
> The `volatile` qualifier is used to tell the compiler that an object may have its value changed at any time. This rarely-used qualifier disables certain types of optimizations.
>
> In technical documentation, the `const` and `volatile` qualifiers are often referred to as **cv-qualifiers**. The following terms are also used in the C++ standard:
>
> - A **cv-unqualified** type is a type with no type qualifiers (e.g. `int`).
> - A **cv-qualified** type is a type with one or more type qualifiers applied (e.g. `const int`).
> - A **possibly cv-qualified** type is a type that may be cv-unqualified or cv-qualified.
>
> These terms are not used much outside of technical documentation, so they are listed here for reference, not as something you need to remember.
>
> But at least now you can appreciate this joke from JF Bastien:
>
> - Q: How do you know if a C++ developer is qualified?
> - A: You look at their CV.

5.2 — Literals

Literals are values that are inserted directly into the code. For example:

```
return 5;                    // 5 is an integer literal
bool myNameIsAlex { true };  // true is a boolean literal
double d { 3.4 };            // 3.4 is a double literal
std::cout << "Hello, world!"; // "Hello, world!" is a C-style string literal
```

Literals are sometimes called **literal constants** because their meaning cannot be redefined (`5` always means the integral value 5).

The type of a literal

Just like objects have a type, all literals have a type. The type of a literal is deduced from the literal's value. For example, a literal that is a whole number (e.g. `5`) is deduced to be of type `int`.

By default:

Literal value	Examples	Default literal type	Note
integer value	5, 0, -3	int	
boolean value	true, false	bool	
floating point value	1.2, 0.0, 3.4	double (not float!)	
character	'a', ''	char	
C-style string	"Hello, world!"	const char[14]	see C-style string literals section below

Literal suffixes

If the default type of a literal is not as desired, you can change the type of a literal by adding a suffix. Here are some of the more common suffixes:

Data type	Suffix	Meaning
integral	u or U	unsigned int
integral	l or L	long
integral	ul, uL, Ul, UL, lu, lU, Lu, LU	unsigned long
integral	ll or LL	long long
integral	ull, uLL, Ull, ULL, llu, llU, LLu, LLU	unsigned long long
integral	z or Z	The signed version of std::size_t (C++23)
integral	uz, uZ, Uz, UZ, zu, zU, Zu, ZU	std::size_t (C++23)
floating point	f or F	float
floating point	l or L	long double
string	s	std::string
string	sv	std::string_view

We'll discuss the integral and floating point literals and suffixes more in a moment.

In most cases, suffixes aren't needed (except for `f`).

Related content

The `s` and `sv` suffixes require an additional line of code to use. We cover these further in lessons 5.7 – Introduction to std::string and 5.8 – Introduction to std::string_view.

Additional (rarely used) suffixes exist for complex numbers and chrono (time) literals. These are documented here.

For advanced readers

Excepting the `f` suffix, suffixes are most often used in cases where type deduction is involved. See 10.8 – Type deduction for objects using the auto keyword and 13.14 – Class template argument deduction (CTAD) and deduction guides.

Suffix casing

Most of the suffixes are not case sensitive. The exceptions are:

- `s` and `sv` must be lower case.
- Two consecutive `l` or `L` characters must have the same casing (`lL` and `Ll` are not accepted).

Because lower-case `L` can look like numeric `1` in some fonts, some developers prefer to use upper-case literals. Others use lower case suffixes except for `L`.

Best practice

Prefer literal suffix L (upper case) over l (lower case).

Integral literals

You generally won't need to use suffixes for integral literals, but here are examples:

```
#include <iostream>

int main()
{
    std::cout << 5 << '\n';  // 5 (no suffix) is type int (by default)
    std::cout << 5L << '\n'; // 5L is type long
    std::cout << 5u << '\n'; // 5u is type unsigned int

    return 0;
}
```

In most cases, it's fine to use non-suffixed `int` literals, even when initializing non-`int` types:

```
int main()
{
    int a { 5 };          // ok: types match
    unsigned int b { 6 }; // ok: compiler will convert int value 6 to unsigned int value 6
    long c { 7 };         // ok: compiler will convert int value 7 to long value 7

    return 0;
}
```

In such cases, the compiler will convert the int literal to the appropriate type.

In the first case, `5` is already an `int` by default, so the compiler can use this value directly to initialize `int` variable `a`. In the second case, `int` value

`6` doesn't match the type of `unsigned int b`. The compiler will convert int value `6` to `unsigned int` value `6`, and then use that as an initializer for `b`. In the third case, `int` value `7` doesn't match the type of `long c`. The compiler will convert int value `7` to `long` value `7`, and then use that as an initializer for `c`.

Floating point literals

By default, floating point literals have a type of `double`. To make them `float` literals instead, the `f` (or `F`) suffix should be used:

```
#include <iostream>

int main()
{
    std::cout << 5.0 << '\n';  // 5.0 (no suffix) is type double (by default)
    std::cout << 5.0f << '\n'; // 5.0f is type float

    return 0;
}
```

New programmers are often confused about why the following causes a compiler warning:

```
float f { 4.1 }; // warning: 4.1 is a double literal, not a float literal
```

Because `4.1` has no suffix, the literal has type `double`, not `float`. When the compiler determines the type of a literal, it doesn't care what you're doing with the literal (e.g. in this case, using it to initialize a `float` variable). Since the type of the literal (`double`) doesn't match the type of the variable it is being used to initialize (`float`), the literal value must be converted to a `float` so it can then be used to initialize variable `f`. Converting a value from a `double` to a `float` can result in a loss of precision, so the compiler will issue a warning.

The solution here is one of the following:

```
float f { 4.1f }; // use 'f' suffix so the literal is a float and matches variable type of float
double d { 4.1 }; // change variable to type double so it matches the literal type double
```

Scientific notation for floating point literals

There are two different ways to write floating-point literals.

1. In standard notation, we write a number with a decimal point:

```
double pi { 3.14159 }; // 3.14159 is a double literal in standard notation
double d { -1.23 };    // the literal can be negative
double why { 0. };     // syntactically acceptable, but avoid this because it's hard to see the decimal point
(prefer 0.0)
```

2. In scientific notation, we add an `e` to represent the exponent:

```
double avogadro { 6.02e23 }; // 6.02 x 10^23 is a double literal in scientific notation
double protonCharge { 1.6e-19 }; // charge on a proton is 1.6 x 10^-19
```

String literals

In programming, a **string** is a collection of sequential characters used to represent text (such as names, words, and sentences).

The very first C++ program you wrote probably looked something like this:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!";
    return 0;
}
```

`"Hello, world!"` is a string literal. String literals are placed between double quotes to identify them as strings (as opposed to char literals, which are placed between single quotes).

Because strings are commonly used in programs, most modern programming languages include a fundamental string data type. For historical reasons, strings are not a fundamental type in C++. Rather, they have a strange, complicated type that is hard to work with (we'll cover how/why in a future lesson, once we've covered more fundamentals required to explain how they work). Such strings are often called **C strings** or **C-style strings**, as they are inherited from the C-language.

There are two non-obvious things worth knowing about C-style string literals.

1. All C-style string literals have an implicit null terminator. Consider a string such as `"hello"`. While this C-style string appears to only have five characters, it actually has six: `'h'`, `'e'`, `'l'`, `'l'`, `'o'`, and `'\0'` (a character with ASCII code 0). This trailing '\0' character is a special character called a **null terminator**, and it is used to indicate the end of the string. A string that ends with a null terminator is called a **null-terminated string**.

> **For advanced readers**
>
> This is the reason the string `"Hello, world!"` has type `const char[14]` rather than `const char[13]` – the hidden null terminator counts as a character.
>
> The reason for the null-terminator is also historical: it can be used to determine where the string ends.

2. Unlike most other literals (which are values, not objects), C-style string literals are const objects that are created at the start of the program and are guaranteed to exist for the entirety of the program. This fact will become important in a few lessons, when we discuss `std::string_view`.

> **Key insight**
>
> C-style string literals are const objects that are created at the start of the program and are guaranteed to exist for the entirety of the program.
>
> Unlike C-style string literals, `std::string` and `std::string_view` literals create temporary objects. These temporary objects must be used immediately, as they are destroyed at the end of the full expression in which they are created.

> **Related content**
>
> We discuss `std::string` and `std::string_view` literals in lesson 5.7 – Introduction to std::string and 5.8 – Introduction to std::string_view respectively.

Magic numbers

A **magic number** is a literal (usually a number) that either has an unclear meaning or may need to be changed later.

Here are two statements showing examples of magic numbers:

```
const int maxStudentsPerSchool{ numClassrooms * 30 };
setMax(30);
```

What do the literals `30` mean in these contexts? In the former, you can probably guess that it's the number of students per class, but it's not immediately obvious. In the latter, who knows. We'd have to go look at the function to know what it does.

In complex programs, it can be very difficult to infer what a literal represents, unless there's a comment to explain it.

Using magic numbers is generally considered bad practice because, in addition to not providing context as to what they are being used for, they pose problems if the value needs to change. Let's assume that the school buys new desks that allow them to raise the class size from 30 to 35, and our program needs to reflect that.

To do so, we need to update one or more literal from `30` to `35`. But which literals? The `30` in the initializer of `maxStudentsPerSchool` seems obvious. But what about the `30` used as an argument to `setMax()`? Does that `30` have the same meaning as the other `30`? If so, it should be updated. If not, it should be left alone, or we might break our program somewhere else. If you do a global search-and-replace, you might inadvertently update the argument of `setMax()` when it wasn't supposed to change. So you have to look through all the code for every instance of the literal `30` (of which there might be hundreds), and then make an individual determination as to whether it needs to change or not. That can be seriously time consuming (and error prone).

Fortunately, both the lack of context and the issues around updating can be easily addressed by using symbolic constants:

```
const int maxStudentsPerClass { 30 };
const int totalStudents{ numClassrooms * maxStudentsPerClass }; // now obvious what this 30 means

const int maxNameLength{ 30 };
setMax(maxNameLength); // now obvious this 30 is used in a different context
```

The name of the constant provides context, and we only need to update a value in one place to make a change to the value across our entire program.

Note that magic numbers aren't always numbers – they can also be text (e.g. names) or other types:

```
int main()
{
    printAppWelcome("MyCalculator"); // bad: app name may be used in other places or change in the future
}
```

Literals used in obvious contexts that are unlikely to change are typically not considered magic. The values `-1`, `0`, `0.0`, and `1` are often used in such contexts:

```
int idGenerator { 0 };         // okay: we're starting our id generator with value 0
idGenerator = idGenerator + 1; // okay: we're just incrementing our generator
```

Other numbers may also be obvious in context (and thus, not considered magic):

```
int kmtoM(int km)
{
    return km * 1000; // okay: it's obvious 1000 is a conversion factor
}
```

Sequential integral ids are also generally not considered magic:

```
int main()
{
    // okay: these are just sequential ids/counts
    printPlayerInfo(1); // `1` would not really benefit from being named `player1` instead
    printPlayerInfo(2);
}
```

> **Best practice**
>
> Avoid magic numbers in your code (use constexpr variables instead, see lesson 5.6 – Constexpr variables).

5.3 — Numeral systems (decimal, binary, hexadecimal, and octal)

> **Author's note**
>
> This lesson is optional.
>
> Future lessons reference hexadecimal numbers, so you should at least have a passing familiarity with the concept before proceeding.

In everyday life, we count using **decimal** numbers, where each numerical digit can be 0, 1, 2, 3, 4, 5, 6, 7, 8, or 9. Decimal is also called "base 10", because there are 10 possible digits (0 through 9). In this system, we count like this: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, … By default, numbers in C++ programs are assumed to be decimal.

```
int x { 12 }; // 12 is assumed to be a decimal number
```

In **binary**, there are only 2 digits: 0 and 1, so it is called "base 2". In binary, we count like this: 0, 1, 10, 11, 100, 101, 110, 111, … To make them easier to read, longer binary numbers are often space-separated into groups of 4 digits (e.g. 1101 0100).

Decimal and binary are two examples of **numeral systems**, which is a fancy name for a collection of symbols (e.g. digits) used to represent numbers. There are 4 main numeral systems available in C++. In order of popularity, these are: decimal (base 10), binary (base 2), hexadecimal (base 16), and octal (base 8).

Nomenclature

In both decimal and binary, the numbers `0` and `1` have the same meaning. In both systems, we call these numbers "zero" and "one".

But what about the number `10`? `10` is the number that occurs after the last single-digit number in the number system. In decimal, `10` is equal to nine plus one. We call this number "ten".

In binary, `10` uses the same digits, but is equal to one plus one (the equivalent of two in decimal). It would be confusing to call binary `10` "ten", because "ten" is nine plus one, and this `10` is one plus one.

Because of this, the names "ten", "eleven", "twelve", etc… are typically reserved for decimal numbers. In non-decimal number systems, we prefer to call those numbers one-zero, one-one, one-two, etc… Binary 101 isn't "one hundred and one", it's "one-zero-one".

Octal and hexadecimal literals

Octal is base 8 – that is, the only digits available are: 0, 1, 2, 3, 4, 5, 6, and 7. In Octal, we count like this: 0, 1, 2, 3, 4, 5, 6, 7, 10, 11, 12, … (note: no 8 and 9, so we skip from 7 to 10).

Decimal	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
Octal	0	1	2	3	4	5	6	7	10	11	12	13

To use an octal literal, prefix your literal with a 0 (zero):

```
#include <iostream>

int main()
{
    int x{ 012 }; // 0 before the number means this is octal
    std::cout << x << '\n';
    return 0;
}
```

This program prints:

```
10
```

Why 10 instead of 12? Because numbers are output in decimal by default, and 12 octal = 10 decimal.

Octal is hardly ever used, and we recommend you avoid it.

Hexadecimal is base 16. In hexadecimal, we count like this: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11, 12, …

Decimal	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11

You can also use lower case letters (though upper case is more common).

To use a hexadecimal literal, prefix your literal with `0x`:

```
#include <iostream>

int main()
{
    int x{ 0xF }; // 0x before the number means this is hexadecimal
    std::cout << x << '\n';
    return 0;
}
```

This program prints:

```
15
```

You can also use a `0X` prefix, but `0x` is conventional because its easier to read.

Numeral systems tables

Here are the four numeral systems lined up, to make it easier to see how each progresses:

```
Decimal         0     1     2     3     4     5     6     7     8     9    10    11    12    13    14    15
Binary          0     1    10    11   100   101   110   111  1000  1001  1010  1011  1100  1101  1110  1111
Octal           0     1     2     3     4     5     6     7    10    11    12    13    14    15    16    17
Hexadecimal     0     1     2     3     4     5     6     7     8     9     A     B     C     D     E     F

Decimal        16    17    18    19    20    21    22    23    24    25    26    27    28    29    30    31
Binary      10000 10001 10010 10011 10100 10101 10110 10111 11000 11001 11010 11011 11100 11101 11110 11111
Octal          20    21    22    23    24    25    26    27    30    31    32    33    34    35    36    37
Hexadecimal    10    11    12    13    14    15    16    17    18    19    1A    1B    1C    1D    1E    1F
```

Each of these rows contains the same pattern: The rightmost digit is incremented from 0 to (base-1). When the digit reaches (base), it is reset to 0 and the digit to the left is incremented by 1. If this left digit reaches (base), it is reset to 0 and the digit to the left of that one is incremented by 1. And so on…

Using hexadecimal to represent binary

Because there are 16 different values for a hexadecimal digit, we can say that a single hexadecimal digit encompasses 4 bits. Consequently, a pair of hexadecimal digits can be used to exactly represent a full byte.

Hexadecimal	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Consider a 32-bit integer with binary value 0011 1010 0111 1111 1001 1000 0010 0110. Because of the length and repetition of digits, that's not easy to read. In hexadecimal, this same value would be: 3A7F 9826, which is much more concise. For this reason, hexadecimal values are often used to represent

memory addresses or raw data in memory (whose type isn't known).

Binary literals

Prior to C++14, there is no support for binary literals. However, hexadecimal literals provide us with a useful workaround (that you may still see in existing code bases):

```
#include <iostream>

int main()
{
    int bin{};    // assume 16-bit ints
    bin = 0x0001; // assign binary 0000 0000 0000 0001 to the variable
    bin = 0x0002; // assign binary 0000 0000 0000 0010 to the variable
    bin = 0x0004; // assign binary 0000 0000 0000 0100 to the variable
    bin = 0x0008; // assign binary 0000 0000 0000 1000 to the variable
    bin = 0x0010; // assign binary 0000 0000 0001 0000 to the variable
    bin = 0x0020; // assign binary 0000 0000 0010 0000 to the variable
    bin = 0x0040; // assign binary 0000 0000 0100 0000 to the variable
    bin = 0x0080; // assign binary 0000 0000 1000 0000 to the variable
    bin = 0x00FF; // assign binary 0000 0000 1111 1111 to the variable
    bin = 0x00B3; // assign binary 0000 0000 1011 0011 to the variable
    bin = 0xF770; // assign binary 1111 0111 0111 0000 to the variable

    return 0;
}
```

In C++14 onward, we can use binary literals by using the `0b` prefix:

```
#include <iostream>

int main()
{
    int bin{};        // assume 16-bit ints
    bin = 0b1;        // assign binary 0000 0000 0000 0001 to the variable
    bin = 0b11;       // assign binary 0000 0000 0000 0011 to the variable
    bin = 0b1010;     // assign binary 0000 0000 0000 1010 to the variable
    bin = 0b11110000; // assign binary 0000 0000 1111 0000 to the variable

    return 0;
}
```

Digit separators

Because long literals can be hard to read, C++14 also adds the ability to use a quotation mark (`'`) as a digit separator.

```
#include <iostream>

int main()
{
    int bin { 0b1011'0010 };  // assign binary 1011 0010 to the variable
    long value { 2'132'673'462 }; // much easier to read than 2132673462

    return 0;
}
```

Also note that the separator can not occur before the first digit of the value:

```
int bin { 0b'1011'0010 };  // error: ' used before first digit of value
```

Digit separators are purely visual and do not impact the literal value in any way.

Outputting values in decimal, octal, or hexadecimal

By default, C++ outputs values in decimal. However, you can change the output format via use of the `std::dec`, `std::oct`, and `std::hex` I/O manipulators:

```
#include <iostream>

int main()
{
    int x { 12 };
    std::cout << x << '\n'; // decimal (by default)
    std::cout << std::hex << x << '\n'; // hexadecimal
    std::cout << x << '\n'; // now hexadecimal
    std::cout << std::oct << x << '\n'; // octal
    std::cout << std::dec << x << '\n'; // return to decimal
    std::cout << x << '\n'; // decimal

    return 0;
}
```

This prints:

```
12
c
c
14
12
12
```

Note that once applied, the I/O manipulator remains set for future output until it is changed again.

Outputting values in binary

Outputting values in binary is a little harder, as `std::cout` doesn't come with this capability built-in. Fortunately, the C++ standard library includes a type called `std::bitset` that will do this for us (in the `<bitset>` header).

To use `std::bitset`, we can define a `std::bitset` variable and tell `std::bitset` how many bits we want to store. The number of bits must be a

compile-time constant. `std::bitset` can be initialized with an integral value (in any format, including decimal, octal, hex, or binary).

```
#include <bitset> // for std::bitset
#include <iostream>

int main()
{
    // std::bitset<8> means we want to store 8 bits
    std::bitset<8> bin1{ 0b1100'0101 }; // binary literal for binary 1100 0101
    std::bitset<8> bin2{ 0xC5 }; // hexadecimal literal for binary 1100 0101

    std::cout << bin1 << '\n' << bin2 << '\n';
    std::cout << std::bitset<4>{ 0b1010 } << '\n'; // create a temporary std::bitset and print it

    return 0;
}
```

This prints:

```
11000101
11000101
1010
```

In the above code, this line:

```
std::cout << std::bitset<4>{ 0b1010 } << '\n'; // create a temporary std::bitset and print it
```

creates a temporary (unnamed) `std::bitset` object with 4 bits, initializes it with binary literal `0b1010`, prints the value in binary, and then discards the temporary object.

> **Related content**
>
> We cover std::bitset in more detail in lesson O.1 – Bit flags and bit manipulation via std::bitset.

Outputting values in binary using the Format / Print Library Advanced

In C++20 and C++23, we have better options for printing binary via the new Format Library (C++20) and Print Library (C++23):

```
#include <format> // C++20
#include <iostream>
#include <print> // C++23

int main()
{
    std::cout << std::format("{:b}\n", 0b1010);  // C++20, {:b} formats the argument as binary digits
    std::cout << std::format("{:#b}\n", 0b1010); // C++20, {:#b} formats the argument as 0b-prefixed binary d

    std::println("{:b} {:#b}", 0b1010, 0b1010);  // C++23, format/print two arguments (same as above) and a
newline

    return 0;
}
```

This prints:

```
1010
0b1010
1010 0b1010
```

5.4 — The as-if rule and compile-time optimization

Introduction to optimization

In programming, **optimization** is the process of modifying software to make it work more efficiently (e.g. to run faster, or use fewer resources). Optimization can have a huge impact on the overall performance level of an application.

Some types of optimization are typically done by hand. A program called a **profiler** can be used to see how long various parts of the program are taking to run, and which are impacting overall performance. The programmer can then look for ways to alleviate those performance issues. Because hand-optimization is slow, programmers typically focuses on making high-level improvements that will have a large impact (such as choosing more performant algorithms, optimizing data storage and access, reducing resource utilization, parallelizing tasks, etc…)

Other kinds of optimization can be performed automatically. A program that optimizes another program is called an **optimizer**. Optimizers typically work at a low-level, looking for ways to improve statements or expressions by rewriting, reordering, or eliminating them. For example, when you write `i = i * 2;`, the optimizer might rewrite this as `i *= 2;`, `i += i;`, or `i <<= 1;`. For integral values, all of these produce the same result, but one might be faster than the others on a given architecture. A programmer would probably not know which is the most performant choice (and the answer might vary based on architecture), but an optimizer for a given system would. Individual low-level optimizations may only yield small performance gains, but their cumulative effect can result in a significant performance improvement overall.

Modern C++ compilers are optimizing compilers, meaning they are capable of automatically optimizing your programs as part of the compilation process. Just like the preprocessor, these optimizations do not modify your source code files – rather, they are applied transparently as part of the compilation process.

> **Key insight**
>
> Optimizing compilers allow programmers to focus on writing code that is readable and maintainable without sacrificing performance.

Because optimization involves some tradeoffs (we'll discuss this at the bottom of the lesson), compilers typically support multiple optimization levels that determine whether they optimize, how aggressively they optimize, and what kind of optimizations they prioritize (e.g. speed vs size).

Most compilers default to no optimization, so if you're using a command-line compiler, you'll need to enable optimization yourself. If you're using an IDE, the IDE will likely automatically configure release builds to enable optimization and debug builds to disable optimization.

> **For gcc and Clang users**
>
> See 0.9 – Configuring your compiler: Build configurations for information on how to enable optimization.

The as-if rule

In C++, compilers are given a lot of leeway to optimize programs. The **as-if rule** says that the compiler can modify a program however it likes in order to produce more optimized code, so long as those modifications do not affect a program's "observable behavior".

> **For advanced readers**
>
> There is one notable exception to the as-if rule: unnecessary calls to a copy (or move) constructor can be elided (omitted) even if those constructors have observable behavior. We cover this topic in lesson 14.15 – Class initialization and copy elision.

Modern compilers employ a variety of different techniques in order to optimize a program effectively. Which techniques can be applied depends on the program and the quality of the compiler and optimizer.

> **Related content**
>
> Wikipedia has list of specific techniques that compilers use.

An optimization opportunity

Consider the following short program:

```
#include <iostream>

int main()
{
    int x { 3 + 4 };
    std::cout << x << '\n';

    return 0;
}
```

The output is straightforward:

```
7
```

However, there's an interesting optimization possibility hidden within.

If this program were compiled exactly as it was written (with no optimizations), the compiler would generate an executable that calculates the result of `3 + 4` at runtime (when the program is run). If the program were executed a million times, `3 + 4` would be evaluated a million times, and the resulting value of `7` produced a million times.

Because the result of `3 + 4` never changes (it is always `7`), re-calculating this result every time the program is run is wasteful.

Compile-time evaluation

Modern C++ compilers are capable of fully or partially evaluating certain expressions at compile-time (rather than at runtime). When the compiler fully or partially evaluates an expression at compile-time, this is called **compile-time evaluation**.

> **Key insight**
>
> Compile-time evaluation allows the compiler to do work at compile-time that would otherwise be done at runtime. Because such expressions no longer need to be evaluated at runtime, the resulting executables are faster and smaller (at the cost of slightly slower compilation times).

For illustrative purposes, in this lesson we will look at some simple optimization techniques that make use of compile-time evaluation. Then, we'll continue our discussion of compile-time evaluation in subsequent lessons.

Constant folding

One of the original forms of compile-time evaluation is called "constant folding". **Constant folding** is an optimization technique where the compiler replaces expressions that have literal operands with the result of the expression. Using constant folding, the compiler would recognize that the expression `3 + 4` has constant operands, and then replace the expression with the result `7`.

The result would be equivalent to the following:

```
#include <iostream>

int main()
{
    int x { 7 };
    std::cout << x << '\n';

    return 0;
}
```

This program produces the same output (`7`) as the prior version, but the resulting executable no longer needs to spend CPU cycles calculating `3 + 4` at

runtime!

Constant folding can also be applied to subexpressions, even when the full expression must execute at runtime.

```
#include <iostream>

int main()
{
    std::cout << 3 + 4 << '\n';

    return 0;
}
```

In the above example, `3 + 4` is a subexpression of the full expression `std::cout << 3 + 4 << '\n';`. The compiler can optimize this to `std::cout << 7 << '\n';`.

Constant propagation

The following program contains another optimization opportunity:

```
#include <iostream>

int main()
{
    int x { 7 };
    std::cout << x << '\n';

    return 0;
}
```

When `x` is initialized, the value `7` will be stored in the memory allocated for `x`. Then on the next line, the program will go out to memory again to fetch the value `7` so it can be printed. This requires two memory access operations (one to store the value, and one to fetch it).

Constant propagation is an optimization technique where the compiler replaces variables known to have constant values with their values. Using constant propagation, the compiler would realize that `x` always has the constant value `7`, and replace any use of variable `x` with the value `7`.

The result would be equivalent to the following:

```
#include <iostream>

int main()
{
    int x { 7 };
    std::cout << 7 << '\n';

    return 0;
}
```

This removes the need for the program to go out to memory to fetch the value of `x`.

Constant propagation may produce a result that can then be optimized by constant folding:

```
#include <iostream>

int main()
{
    int x { 7 };
    int y { 3 };
    std::cout << x + y << '\n';

    return 0;
}
```

In this example, constant propagation would transform `x + y` into `7 + 3`, which can then be constant folded into the value `10`.

Dead code elimination

Dead code elimination is an optimization technique where the compiler removes code that may be executed but has no effect on the program's behavior.

Back to a prior example:

```
#include <iostream>

int main()
{
    int x { 7 };
    std::cout << 7 << '\n';

    return 0;
}
```

In this program, variable `x` is defined and initialized, but it is never used anywhere, so it has no effect on the program's behavior. Dead code elimination would remove the definition of `x`.

The result would be equivalent to the following:

```
#include <iostream>

int main()
{
    std::cout << 7 << '\n';

    return 0;
}
```

When a variable is removed from a program because it is no longer needed, we say the variable has been **optimized out** (or **optimized away**).

Compared to the original version, this optimized version no longer requires runtime calculation expression `3 + 4`, nor does it require two memory access operations (one to initialize variable `x` and one to read the value from `x`). This means the program will be both smaller and faster.

Const variables are easier to optimize

In some cases, there are simple things we can do to help the compiler optimize more effectively.

Constant propagation can be challenging for the compiler. In the section on constant propagation, we offered this example:

```
#include <iostream>

int main()
{
    int x { 7 };
    std::cout << x << '\n';

    return 0;
}
```

Since `x` is defined as a non-const variable, in order to apply this optimization, the compiler must realize that the value of `x` actually doesn't change (even though it could). Whether the compiler is capable of doing so comes down to how complex the program is and how sophisticated the compiler's optimization routines are.

We can help the compiler optimize more effectively by using constant variables wherever possible. For example:

```
#include <iostream>

int main()
{
    const int x { 7 }; // x is now const
    std::cout << x << '\n';

    return 0;
}
```

Because `x` is now const, the compiler has a guarantee that `x` can't be changed after initialization. This makes it more likely the compiler will apply constant propagation, and then optimize the variable out entirely.

> **Key insight**
>
> Using const variables can help the compiler optimize more effectively.

Optimization can make programs harder to debug

If optimization makes our programs faster, why isn't it turned on by default?

When the compiler optimizes a program, the result is that variables, expressions, statements, and function calls may be rearranged, modified, replaced, or removed entirely. Such changes can make it hard to debug a program effectively.

At runtime, it can be hard to debug compiled code that no longer correlates very well with the original source code. For example, if you try to watch a variable that has been optimized out, the debugger won't be able to locate the variable. If you try to step into a function that has been optimized away, the debugger will simply skip over it. So if you are debugging your code and the debugger is behaving strangely, this is the most likely reason.

At compile-time, we have little visibility and few tools to help us understand what the compiler is even doing. If a variable or expression is replaced with a value, and that value is wrong, how do we even go about debugging the issue? This is an ongoing challenge.

To help minimize such issues, debug builds will typically leave optimizations turned off, so that the compiled code will more closely match the source code.

> **Author's note**
>
> Compile-time debugging is an underdeveloped area. As of C++23, there are a number of papers under consideration for future language standards (such as this one) that (if approved) will add capabilities to the language that will help.

Nomenclature: Compile-time constants vs runtime constants

Constants in C++ are sometimes divided into two informal categories.

A **compile-time constant** is a constant whose value is known at compile-time. Examples include:

- Literals.
- Constant objects whose initializers are compile-time constants.

A **runtime constant** is a constant whose value is determined in a runtime context. Examples include:

- Constant function parameters.
- Constant objects whose initializers are non-constants or runtime constants.

For example:

```
#include <iostream>

int five()
{
    return 5;
}

int pass(const int x) // x is a runtime constant
{
    return x;
}

int main()
{
    // The following are non-constants:
    [[maybe_unused]] int a { 5 };

    // The following are compile-time constants:
    [[maybe_unused]] const int b { 5 };
    [[maybe_unused]] const double c { 1.2 };
    [[maybe_unused]] const int d { b };       // b is a compile-time constant

    // The following are runtime constants:
    [[maybe_unused]] const int e { a };       // a is non-const
    [[maybe_unused]] const int f { e };       // e is a runtime constant
    [[maybe_unused]] const int g { five() };  // return value isn't known until runtime
    [[maybe_unused]] const int h { pass(5) }; // return value isn't known until runtime

    return 0;
}
```

Although you will encounter these terms out in the wild, in C++ these definitions are not all that useful:

- Some runtime constants (and even non-constants) can be evaluated at compile-time for optimization purposes (under the as-if rule).
- Some compile-time constants (e.g. `const double d { 1.2 };`) cannot be used in compile-time features (as defined by the language standard). We'll discuss this more in lesson 5.5 – Constant expressions.

For this reason, we recommend avoiding these terms. We'll discuss the nomenclature that you should use instead in the next lesson.

> **Author's note**
>
> We are in the process of phasing these terms out of future articles.

5.5 — Constant expressions

In lesson 1.10 – Introduction to expressions, we introduced expressions. By default, expressions evaluate at runtime. And in some cases, they must do so:

```
std::cin >> x;
std::cout << 5 << '\n';
```

Because input and output can't be performed at compile time, the expressions above are required to evaluate at runtime.

In prior lesson 5.4 – The as-if rule and compile-time optimization, we discussed the as-if rule, and how the compiler can optimize programs by shifting work from runtime to compile-time. Under the as-if rule, the compiler may choose whether to evaluate certain expressions at runtime or compile-time:

```
const double x { 1.2 };
const double y { 3.4 };
const double z { x + y }; // x + y may evaluate at runtime or compile-time
```

The expression `x + y` would normally evaluate at runtime, but since the value of `x` and `y` are known at compile-time, the compiler may opt to perform compile-time evaluation instead and initialize `z` with the compile-time calculated value `4.6`.

In a few other cases, the C++ language requires an expression that can be evaluated at compile-time. For example, constexpr variables require an initializer that can be evaluated at compile-time:

```
int main()
{
    constexpr int x { expr }; // Because variable x is constexpr, expr must be evaluatable at compile-time
}
```

In cases where a constant expression is required but one is not provided, the compiler will error and halt compilation.

We'll discuss constexpr variables in the next lesson (5.6 – Constexpr variables), when we cover constexpr variables.

> **For advanced readers**
>
> A few common cases where a compile-time evaluatable expression is required:
>
> - The initializer of a constexpr variable (5.6 – Constexpr variables).
> - A non-type template argument (11.9 – Non-type template parameters).
> - The defined length of a `std::array` (17.1 – Introduction to std::array) or a C-style array (17.7 – Introduction to C-style arrays).

In this lesson, we'll explore more of C++'s capabilities around compile-time evaluation, and look at how C++ differentiates this last case from the prior two cases.

The benefits of compile-time programming

While the as-if rule is great for improving performance, it leaves us reliant on the sophistication of the compiler to actually determine what can evaluate at compile-time. This means if there is a section of code we really want to execute at compile-time, it may or may not. That same code compiled on a different platform, or with a different compiler, or using different compilation options, or slightly modified, may produce a different result. Because the as-if rule is applied invisibly to us, we get no feedback from the compiler on what portions of code it decided to evaluate at compile-time, or why. Code we desire to be evaluated at compile-time may not even be eligible (due to a typo or misunderstanding), and we may never know.

To improve upon this situation, the C++ language provides ways for us to be explicit about what parts of code we want to execute at compile-time. The use of language features that result in compile-time evaluation is called **compile-time programming**.

These features can help improve software in a number of areas:

- **Performance**: Compile-time evaluation makes our programs smaller and faster. The more code we can ensure is capable of evaluating at compile-time, the more performance benefit we'll see.

- **Versatility**: We can always use such code in places that require a compile-time value. Code that relies on the as-if rule to evaluate at compile-time can't be used in such places (even if the compiler opts to evaluate that code at compile-time) – this decision was made so that code that compiles today won't stop compiling tomorrow, when the compiler decides to optimize differently.
- **Predictability**: We can have the compiler halt compilation if it determines that code cannot be executed at compile-time (rather than silently opting to have that code evaluate at runtime instead). This allows us to ensure a section of code we really want to execute at compile-time will.
- **Quality**: We can have the compiler reliably detect certain kinds of programming errors at compile-time, and halt the build if it encounters them. This is much more effective than trying to detect and gracefully handle those same errors at runtime.
- **Quality**: Perhaps most importantly, undefined behavior is not allowed at compile-time. If we do something that causes undefined behavior at compile-time, the compiler should halt the build and ask us to fix it. Note that this is a hard problem for compilers, and they may not catch all cases.

To summarize, compile-time evaluation allows us to write programs that are both more performant and of higher quality (more secure and less buggy)! So while compile-time evaluation does add some additional complexity to the language, the benefits can be substantial.

The following C++ features are the most foundational to compile-time programming:

- Constexpr variables (discussed in upcoming lesson 5.6 – Constexpr variables).
- Constexpr functions (discussed in upcoming lesson F.1 – Constexpr functions).
- Templates (introduced in lesson 11.6 – Function templates).
- static_assert (discussed in lesson 9.6 – Assert and static_assert).

All of these features have one thing in common: they make use of constant expressions.

Constant expressions

Perhaps surprisingly, the C++ standard barely mentions "compile-time" at all. Instead, the standard defines a "constant expression", which is an expression that must be evaluatable at compile-time, along with rules that determine how the compiler should handle these expressions. Constant expressions form the backbone of compile-time evaluation in C++.

In lesson 1.10 – Introduction to expressions, we defined an expression as "a non-empty sequence of literals, variables, operators, and function calls". A **constant expression** is a non-empty sequence of literals, constant variables, operators, and function calls, all of which must be evaluatable at compile-time. The key difference is that in a constant expression, each part of the expression must be evaluatable at compile-time.

> **Key insight**
>
> In a constant expression, each part of the expression must be evaluatable at compile-time.

An expression that is not a constant expression is often called a non-constant expression, and may informally be called a **runtime expression** (as such expressions typically evaluate at runtime).

> **Optional reading**
>
> The C++20 language standard (in section [expr.const]) states "Constant expressions can be evaluated during translation". As we covered in lesson 2.10 – Introduction to the preprocessor, translation is the whole process of building a program (that includes preprocessing, compiling, and linking). Therefore, in a compiled program, constant expressions can be evaluated as part of the compilation process. In an interpreted program, translation happens at runtime.
>
> Since C++ programs are typically compiled, we'll proceed under the assumption that constant expressions can be evaluated at compile-time.

What can be in a constant expression?

> **Author's note**
>
> In technical terms, constant expressions are quite complex. In this section, we'll go into a little bit deeper into what they can and can't contain. You do not need to remember most of this. If a constant expression is required somewhere and you do not provide one, the compiler will happily point out your mistake, and you can fix it at that point.

Most commonly, constant expressions contain the following:

- Literals (e.g. `5`, `1.2`)
- Most operators with constant expression operands (e.g. `3 + 4`, `2 * sizeof(int)`).
- Const integral variables with a constant expression initializer (e.g. `const int x { 5 };`). This is a historical exception – in modern C++, constexpr variables are preferred.
- Constexpr variables (discussed in upcoming lesson 5.6 – Constexpr variables).
- Constexpr function calls with constant expression arguments (see F.1 – Constexpr functions).

> **For advanced readers**
>
> Constant expressions can also contain:
>
> - Non-type template parameters (see 11.9 – Non-type template parameters).
> - Enumerators (see 13.2 – Unscoped enumerations).
> - Type traits (see the cppreference page for type traits).
> - Constexpr lambda expressions (see 20.6 – Introduction to lambdas (anonymous functions)).

> **Tip**
>
> Notably, the following cannot be used in a constant expression:
>
> - Non-const variables.
> - Const non-integral variables, even when they have a constant expression initializer (e.g. `const double d { 1.2 };`). To use such variables in a constant expression, define them as constexpr variables instead (see lesson 5.6 – Constexpr variables).
> - The return values of non-constexpr functions (even when the return expression is a constant expression).
> - Function parameters (even when the function is constexpr).
> - Operators with operands that are not constant expressions (e.g. `x + y` when `x` or `y` is not a constant expression, or `std::cout << "hello\n"` as `std::cout` is not a constant expression).
> - Operators `new`, `delete`, `throw`, `typeid`, and `operator,` (comma).
>
> An expression containing any of the above is a runtime expression.

> **Related content**
>
> For the precise definition of a constant expression, see the cppreference page for constant expression. Note that a constant expression is defined by what kind of expression it is not. That means we're left to infer what it is. Good luck with that!

Nomenclature

When discussing constant expressions, it is common to use one of two phrasings:

- "X is usable in a constant expression" is often used when emphasizing what X is. e.g. "`5` is usable in a constant expression" emphasizes that the literal `5` can be used in a constant expression.
- "X is a constant expression" is sometimes used when emphasizing that the full expression (consisting of X) is a constant expression. e.g. "`5` is a constant expression" emphasizes that the expression `5` is a constant expression.

The latter can sound awkward when phrased like "literals are constant expressions" (because they are actually values). But it simply means that an expression consisting of a literal is a constant expression.

> **As an aside…**
>
> When constant expressions were defined, `const` integral types were grandfathered in because they were already being treated as constant expressions within the language.
>
> The committee discussed whether `const` non-integral types with a constant expression initializer should also be treated as constant expressions (for consistency with the `const` integral types case). Ultimately, they decided not to, in order to promote more consistent usage of `constexpr`.

Examples of constant and non-constant expressions

In the following program, we look at some expression statements and indicate whether each expression is a constant expression or runtime expression:

```
#include <iostream>

int getNumber()
{
    std::cout << "Enter a number: ";
    int y{};
    std::cin >> y; // can only execute at runtime

    return y;      // this return expression is a runtime expression
}

// The return value of a non-constexpr function is a runtime expression
// even when the return expression is a constant expression
int five()
{
    return 5;      // this return expression is a constant expression
}

int main()
{
    // Literals can be used in constant expressions
    5;                          // constant expression
    1.2;                        // constant expression
    "Hello world!";             // constant expression

    // Most operators that have constant expression operands can be used in constant expressions
    5 + 6;                      // constant expression
    1.2 * 3.4;                  // constant expression
    8 - 5.6;                    // constant expression (even though operands have different types)
    sizeof(int) + 1;            // constant expression (sizeof can be determined at compile-time)

    // The return values of non-constexpr functions can only be used in runtime expressions
    getNumber();                // runtime expression
    five();                     // runtime expression (even though the return expression is a constant
expression)

    // Operators without constant expression operands can only be used in runtime expressions
    std::cout << 5;             // runtime expression (std::cout isn't a constant expression operand)

    return 0;
}
```

In the following snippet, we define a bunch of variables, and indicate whether they can be used in constant expressions:

```
        // Const integral variables with a constant expression initializer can be used in constant expressions:
            const int a { 5 };           // a is usable in constant expressions
            const int b { a };           // b is usable in constant expressions (a is a constant expression per the p
statement)
            const long c { a + 2 };      // c is usable in constant expressions (operator+ has constant expression
operands)

            // Other variables cannot be used in constant expressions (even when they have a constant expression
initializer):
            int d { 5 };                 // d is not usable in constant expressions (d is non-const)
            const int e { d };           // e is not usable in constant expressions (initializer is not a constant
expression)
            const double f { 1.2 };      // f is not usable in constant expressions (not a const integral variable)
```

When constant expressions are evaluated at compile-time

Since constant expressions are always capable of being evaluated at compile-time, you may have assumed that constant expressions will always be evaluated at compile-time. Counterintuitively, this is not the case.

The compiler is only *required* to evaluate constant expressions at compile-time in contexts that *require* a constant expression.

> **Nomenclature**
>
> The technical name for an expression that must be evaluated at compile-time is a **manifestly constant-evaluated expression**. You will likely only encounter this term in technical documentation.

In contexts that do not require a constant expression, the compiler may choose whether to evaluate a constant expression at compile-time or at runtime.

```
        const int x { 3 + 4 }; // constant expression 3 + 4 must be evaluated at compile-time
        int y { 3 + 4 };       // constant expression 3 + 4 may be evaluated at compile-time or runtime
```

Variable `x` has type `const int` and a constant expression initializer, `x` is usable in a constant expression. Its initializer must be evaluated at compile-time (otherwise the value of `x` wouldn't be known at compile-time, and `x` wouldn't be usable in a constant expression). On the other hand, variable `y` is non-const, so `y` is not usable in a constant expression. Even though its initializer is a constant expression, the compiler can decide to evaluate the initializer at compile-time or runtime.

Even when not required to do so, modern compilers will *usually* evaluate a constant expression at compile-time when optimizations are enabled.

> **Key insight**
>
> The compiler is only *required* to evaluate constant expressions at compile-time in contexts that *require* a constant expression. It may or may not do so in other cases.

> **Tip**
>
> The likelihood that an expression is fully evaluated at compile-time can be categorized as follows:
>
> - **Never**: A non-constant expression where the compiler is not able to determine all values at compile-time.
> - **Possibly**: A non-constant expression where the compiler is able to determine all values at compile-time (optimized under the as-if rule).
> - **Likely**: A constant expression used in a context that does not require a constant expression.
> - **Always**: A constant expression used in a context that requires a constant expression.

> **For advanced readers**
>
> So why doesn't C++ require all constant expressions to be evaluated at compile-time? There are at least two good reasons:

> 1. Compile-time evaluation makes debugging harder. If our code has a buggy calculation that is evaluated at compile-time, we have limited tools to diagnose the issue. Allowing non-required constant expressions to be evaluated at runtime (typically when optimizations are turned off) enables runtime debugging of our code. Being able to step through and inspect the state of our programs while they are running can make finding bugs easier.
> 2. To provide the compiler with the flexibility to optimize as it sees fit (or as influenced by compiler options). For example, a compiler might want to offer an option that defers all non-required constant expression evaluation to runtime, in order to improve compile times for developers.

Why compile-time expressions must be constant Optional

You may be wondering why compile-time expressions can only contain constant objects (and operators and functions that can evaluate at compile-time to constants).

Consider the following program:

```
#include <iostream>

int main()
{
    int x { 5 };
    // x is known to the compiler at this point

    std::cin >> x; // read in value of x from user
    // x is no longer known to the compiler

    return 0;
}
```

To start, `x` is initialized with value `5`. The value of `x` is

5.6 — Constexpr variables

In the previous lesson 5.5 – Constant expressions, we defined what a constant expression is, discussed why constant expressions are desirable, and concluded with when constant expressions actually evaluate at compile-time.

In this lesson, we'll take a closer look at how we create variables that can be used in constant expressions in modern C++. We'll also explore our first method for ensuring that code actually executes at compile-time.

The compile-time `const` challenge

In the prior lesson, we noted that one way to create a variable that can be used in a constant expression is to use the `const` keyword. A `const` variable with an integral type and a constant expression initializer can be used in a constant expression. All other `const` variables cannot be used in constant expressions.

However, the use of `const` to create variables that can be used in constant expressions has a few challenges.

First, use of `const` does not make it immediately clear whether the variable is usable in a constant expression or not. In some cases, we can figure it out fairly easily:

```
int a { 5 };       // not const at all
const int b { a }; // clearly not a constant expression (since initializer is non-const)
const int c { 5 }; // clearly a constant expression (since initializer is a constant expression)
```

In other cases, it can be quite difficult:

```
const int d { someVar };    // not obvious whether d is usable in a constant expression or not
const int e { getValue() }; // not obvious whether e is usable in a constant expression or not
```

In the above example, variables `d` and `e` may or may not be usable in a constant expressions, depending on how `someVar` and `getValue()` are defined. That means we have to go inspect the definitions of those initializers and infer what case we're in. And that may not even be sufficient – if `someVar` is const and initialized with a variable or a function call, we'll have to go inspect the definition of its initializer too!

Second, use of `const` does not provide a way to inform the compiler that we require a variable that is usable in a constant expression (and that it should halt compilation if it isn't). Instead, it will just silently create a variable that can only be used in runtime expressions.

Third, the use of `const` to create compile-time constant variables does not extend to non-integral variables. And there are many cases where we would like non-integral variables to be compile-time constants too.

The `constexpr` keyword

Fortunately, we can enlist the compiler's help to ensure we get a compile-time constant variable where we desire one. To do so, we use the `constexpr` keyword (which is shorthand for "constant expression") instead of `const` in a variable's declaration. A **constexpr** variable is always a compile-time constant. As a result, a constexpr variable must be initialized with a constant expression, otherwise a compilation error will result.

For example:

```
#include <iostream>

// The return value of a non-constexpr function is not constexpr
int five()
{
    return 5;
}

int main()
{
    constexpr double gravity { 9.8 }; // ok: 9.8 is a constant expression
    constexpr int sum { 4 + 5 };      // ok: 4 + 5 is a constant expression
    constexpr int something { sum };  // ok: sum is a constant expression

    std::cout << "Enter your age: ";
    int age{};
    std::cin >> age;

    constexpr int myAge { age };      // compile error: age is not a constant expression
    constexpr int f { five() };       // compile error: return value of five() is not constexpr

    return 0;
}
```

Because functions normally execute at runtime, the return value of a function is not constexpr (even when the return expression is a constant expression). This is why `five()` is not a legal initialization value for `constexpr int f`.

> **Related content**
>
> We talk about functions whose return values can be used in constant expressions in lesson F.1 – Constexpr functions.

Additionally, `constexpr` works for variables with non-integral types:

```
constexpr double d { 1.2 }; // d can be used in constant expressions!
```

The meaning of const vs constexpr for variables

For variables:

- `const` means that the value of an object cannot be changed after initialization. The value of the initializer may be known at compile-time or runtime. The const object can be evaluated at runtime.
- `constexpr` means that the object can be used in a constant expression. The value of the initializer must be known at compile-time. The constexpr object can be evaluated at runtime or compile-time.

Constexpr variables are implicitly const. Const variables are not implicitly constexpr (except for const integral variables with a constant expression initializer). Although a variable can be defined as both `constexpr` and `const`, in most cases this is redundant, and we only need to use either `const` or `constexpr`.

Unlike `const`, `constexpr` is not part of an object's type. Therefore a variable defined as `constexpr int` actually has type `const int` (due to the implicit `const` that `constexpr` provides for objects).

> **Best practice**
>
> Any constant variable whose initializer is a constant expression should be declared as `constexpr`.
>
> Any constant variable whose initializer is not a constant expression (making it a runtime constant) should be declared as `const`.

> Caveat: In the future we will discuss some types that are not fully compatible with `constexpr` (including `std::string`, `std::vector`, and other types that use dynamic memory allocation). For constant objects of these types, either use `const` instead of `constexpr`, or pick a different type that is constexpr compatible (e.g. `std::string_view` or `std::array`).

Nomenclature

The term `constexpr` is a portmanteau of "constant expression". This name was picked because constexpr objects (and functions) can be used in constant expressions.

Formally, the keyword `constexpr` applies only to objects and functions. Conventionally, the term `constexpr` is used as shorthand for any constant expression (such as `1 + 2`).

> **Author's note**
>
> Some of the examples on this site were written prior to the best practice to use `constexpr` – as a result, you will note that some examples do not follow the above best practice. We are currently in the process of updating non-compliant examples as we run across them.

> **For advanced readers**
>
> In C and C++, the declaration of an array object (an object can hold multiple values) requires the length of the array (the number of values that it can hold) be known at compile-time (so the compiler can ensure the correct amount of memory is allocated for array objects).
>
> Since literals are known at compile-time, they can be used as an array length:
>
> ```
> int arr[5]; // an array of 5 int values, length of 5 is known at compile-time
> ```
>
> In many cases, it would be preferable to use a symbolic constant as an array length (e.g. to avoid magic numbers and make the array length easier to change if it is used in multiple places). In C, this can be done via a preprocessor macro, or via an enumerator, but not via a const variable (excluding VLA's, which have other downsides). C++, looking to improve on this situation, wanted to allow the use of const variables instead of macros. But the value of variables was generally assumed to be known only at runtime, which made them ineligible to be used as array lengths.
>
> To solve this problem, the C++ language standard added an exemption so that const integral types with a constant expression initializer would be treated as values known at compile-time, and thus be usable as array lengths:
>
> ```
> const int arrLen = 5;
> int arr[arrLen]; // ok: array of 5 ints
> ```
>
> When C++11 introduced constant expressions, it made sense for a const int with a constant expression initializer to be grandfathered into that definition. The committee discussed whether other types should be included as well, but ultimately decided not to.

Const and constexpr function parameters

Normal function calls are evaluated at runtime, with the supplied arguments being used to initialize the function's parameters. Because the initialization of function parameters happens at runtime, this leads to two consequences:

1. `const` function parameters are treated as runtime constants (even when the supplied argument is a compile-time constant).
2. Function parameters cannot be declared as `constexpr`, since their initialization value isn't determined until runtime.

> **Related content**
>
> We discuss functions that can be evaluated at compile-time (and thus be used in constant expressions) below.
>
> C++ also supports a way to pass compile-time constants to a function. We discuss these in lesson 11.9 – Non-type template parameters.

Nomenclature recap

Term	Definition
Compile-time constant	A value or non-modifiable object whose value must be known at compile time (e.g. literals and constexpr variables).
Constexpr	Keyword that declares objects as compile-time constants (and functions that can be evaluated at compile-time). Informally, shorthand for "constant expression".
Constant expression	An expression that contains only compile-time constants and operators/functions that support compile-time evaluation.
Runtime expression	An expression that is not a constant expression.
Runtime constant	A value or non-modifiable object that is not a compile-time constant.

A brief introduction to constexpr functions

A **constexpr function** is a function that can be called in a constant expression. A constexpr function must evaluate at compile-time when the constant expression it is part of must evaluate at compile time (e.g. in the initializer of a constexpr variable). Otherwise, a constexpr function may be evaluated at either compile-time (if eligible) or runtime. To be eligible for compile-time execution, all arguments must be constant expressions.

To make a constexpr function, the `constexpr` keyword is placed in the function declaration before the return type:

```
#include <iostream>

int max(int x, int y) // this is a non-constexpr function
{
    if (x > y)
        return x;
    else
        return y;
}

constexpr int cmax(int x, int y) // this is a constexpr function
{
    if (x > y)
        return x;
    else
        return y;
}

int main()
{
    int m1 { max(5, 6) };            // ok
    const int m2 { max(5, 6) };      // ok
    constexpr int m3 { max(5, 6) };  // compile error: max(5, 6) not a constant expression

    int m4 { cmax(5, 6) };           // ok: may evaluate at compile-time or runtime
    const int m5 { cmax(5, 6) };     // ok: may evaluate at compile-time or runtime
    constexpr int m6 { cmax(5, 6) }; // okay: must evaluate at compile-time

    return 0;
}
```

> **Author's note**
>
> We used to discuss constexpr functions in detail in this chapter, but feedback from readers indicated that the topic was too long and nuanced to present this early in the tutorial series. As a result, we've moved the full discussion back to lesson F.1 – Constexpr functions.
>
> The key thing to take away from this introduction is that a constexpr function may be called in constant expressions.
>
> You will see constexpr functions used in some future examples (where appropriate), but we will not expect you to understand them further or write your own constexpr functions until we've formally covered the topic.

5.7 — Introduction to std::string

In lesson 5.2 – Literals, we introduced C-style string literals:

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!"; // "Hello world!" is a C-style string literal.
    return 0;
}
```

While C-style string literals are fine to use, C-style string variables behave oddly, are hard to work with (e.g. you can't use assignment to assign a C-style string variable a new value), and are dangerous (e.g. if you copy a larger C-style string into the space allocated for a shorter C-style string, undefined behavior will result). In modern C++, C-style string variables are best avoided.

Fortunately, C++ has introduced two additional string types into the language that are much easier and safer to work with: `std::string` and `std::string_view` (C++17). Unlike the types we've introduced previously, `std::string` and `std::string_view` aren't fundamental types (they're class types, which we'll cover in the future). However, basic usage of each is straightforward and useful enough that we'll introduce them here.

Introducing `std::string`

The easiest way to work with strings and string objects in C++ is via the `std::string` type, which lives in the `<string>` header.

We can create objects of type `std::string` just like other objects:

```
#include <string> // allows use of std::string

int main()
{
    std::string name {}; // empty string

    return 0;
}
```

Just like normal variables, you can initialize or assign values to `std::string` objects as you would expect:

```
#include <string>

int main()
{
    std::string name { "Alex" }; // initialize name with string literal "Alex"
    name = "John";               // change name to "John"

    return 0;
}
```

Note that strings can be composed of numeric characters as well:

```
std::string myID{ "45" }; // "45" is not the same as integer 45!
```

In string form, numbers are treated as text, not as numbers, and thus they can not be manipulated as numbers (e.g. you can't multiply them). C++ will not automatically convert strings to integer or floating point values or vice-versa (though there are ways to do so that we'll cover in a future lesson).

String output with `std::cout`

`std::string` objects can be output as expected using `std::cout`:

```
#include <iostream>
#include <string>

int main()
{
    std::string name { "Alex" };
    std::cout << "My name is: " << name << '\n';

    return 0;
}
```

This prints:

```
My name is: Alex
```

Empty strings will print nothing:

```
#include <iostream>
#include <string>

int main()
{
    std::string empty{ };
    std::cout << '[' << empty << ']';

    return 0;
}
```

Which prints:

```
[]
```

`std::string` can handle strings of different lengths

One of the neatest things that `std::string` can do is store strings of different lengths:

```
#include <iostream>
#include <string>

int main()
{
    std::string name { "Alex" }; // initialize name with string literal "Alex"
    std::cout << name << '\n';

    name = "Jason";              // change name to a longer string
    std::cout << name << '\n';

    name = "Jay";                // change name to a shorter string
    std::cout << name << '\n';

    return 0;
}
```

This prints:

```
Alex
Jason
Jay
```

In the above example, `name` is initialized with the string `"Alex"`, which contains five characters (four explicit characters and a null-terminator). We then set `name` to a larger string, and then a smaller string. `std::string` has no problem handling this! You can even store really long strings in a `std::string`.

This is one of the reasons that `std::string` is so powerful.

> **Key insight**
>
> If `std::string` doesn't have enough memory to store a string, it will request additional memory (at runtime) using a form of memory allocation known as dynamic memory allocation. This ability to acquire additional memory is part of what makes `std::string` so flexible, but also comparatively slow.
>
> We cover dynamic memory allocation in a future chapter.

String input with `std::cin`

Using `std::string` with `std::cin` may yield some surprises! Consider the following example:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Enter your full name: ";
    std::string name{};
    std::cin >> name; // this won't work as expected since std::cin breaks on whitespace

    std::cout << "Enter your favorite color: ";
    std::string color{};
    std::cin >> color;

    std::cout << "Your name is " << name << " and your favorite color is " << color << '\n';

    return 0;
}
```

Here's the results from a sample run of this program:

```
Enter your full name: John Doe
Enter your favorite color: Your name is John and your favorite color is Doe
```

Hmmm, that isn't right! What happened? It turns out that when using `operator>>` to extract a string from `std::cin`, `operator>>` only returns characters up to the first whitespace it encounters. Any other characters are left inside `std::cin`, waiting for the next extraction.

So when we used `operator>>` to extract input into variable `name`, only `"John"` was extracted, leaving `" Doe"` inside `std::cin`. When we then used `operator>>` to get extract input into variable `color`, it extracted `"Doe"` instead of waiting for us to input an color. Then the program ends.

Use `std::getline()` to input text

To read a full line of input into a string, you're better off using the `std::getline()` function instead. `std::getline()` requires two arguments: the first is `std::cin`, and the second is your string variable.

Here's the same program as above using `std::getline()`:

```
#include <iostream>
#include <string> // For std::string and std::getline

int main()
{
    std::cout << "Enter your full name: ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // read a full line of text into name

    std::cout << "Enter your favorite color: ";
    std::string color{};
    std::getline(std::cin >> std::ws, color); // read a full line of text into color

    std::cout << "Your name is " << name << " and your favorite color is " << color << '\n';

    return 0;
}
```

Now our program works as expected:

```
Enter your full name: John Doe
Enter your favorite color: blue
Your name is John Doe and your favorite color is blue
```

What the heck is `std::ws`?

In lesson 4.8 – Floating point numbers, we discussed output manipulators, which allow us to alter the way output is displayed. In that lesson, we used the output manipulator function `std::setprecision()` to change the number of digits of precision that `std::cout` displayed.

C++ also supports input manipulators, which alter the way that input is accepted. The `std::ws` input manipulator tells `std::cin` to ignore any leading whitespace before extraction. Leading whitespace is any whitespace character (spaces, tabs, newlines) that occur at the start of the string.

Let's explore why this is useful. Consider the following program:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice{};
    std::cin >> choice;

    std::cout << "Now enter your name: ";
    std::string name{};
    std::getline(std::cin, name); // note: no std::ws here

    std::cout << "Hello, " << name << ", you picked " << choice << '\n';

    return 0;
}
```

Here's some output from this program:

```
Pick 1 or 2: 2
Now enter your name: Hello, , you picked 2
```

This program first asks you to enter 1 or 2, and waits for you to do so. All good so far. Then it will ask you to enter your name. However, it won't actually wait for you to enter your name! Instead, it prints the "Hello" string, and then exits.

When you enter a value using `operator>>`, `std::cin` not only captures the value, it also captures the newline character (`'\n'`) that occurs when you hit the enter key. So when we type `2` and then hit enter, `std::cin` captures the string `"2\n"` as input. It then extracts the value `2` to variable `choice`, leaving the newline character behind for later. Then, when `std::getline()` goes to extract text to `name`, it sees `"\n"` is already waiting in `std::cin`, and figures we must have previously entered an empty string! Definitely not what was intended.

We can amend the above program to use the `std::ws` input manipulator, to tell `std::getline()` to ignore any leading whitespace characters:

```
#include <iostream>
#include <string>

int main()
{
    std::cout << "Pick 1 or 2: ";
    int choice{};
    std::cin >> choice;

    std::cout << "Now enter your name: ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // note: added std::ws here

    std::cout << "Hello, " << name << ", you picked " << choice << '\n';

    return 0;
}
```

Now this program will function as intended.

```
Pick 1 or 2: 2
Now enter your name: Alex
Hello, Alex, you picked 2
```

Best practice

If using `std::getline()` to read strings, use `std::cin >> std::ws` input manipulator to ignore leading whitespace. This needs to be done for each `std::getline()` call, as `std::ws` is not preserved across calls.

Key insight

When extracting to a variable, the extraction operator (`>>`) ignores leading whitespace. It stops extracting when encountering non-leading whitespace.

`std::getline()` does not ignore leading whitespace. If you want it to ignore leading whitespace, pass `std::cin >> std::ws` as the first argument. It stops extracting when encountering a newline.

The length of a `std::string`

If we want to know how many characters are in a `std::string`, we can ask a `std::string` object for its length. The syntax for doing this is different than you've seen before, but is pretty straightforward:

```
#include <iostream>
#include <string>

int main()
{
    std::string name{ "Alex" };
    std::cout << name << " has " << name.length() << " characters\n";

    return 0;
}
```

This prints:

```
Alex has 4 characters
```

Although `std::string` is required to be null-terminated (as of C++11), the returned length of a `std::string` does not include the implicit null-terminator character.

Note that instead of asking for the string length as `length(name)`, we say `name.length()`. The `length()` function isn't a normal standalone function – it's a special type of function that is nested within `std::string` called a *member function*. Because the `length()` member function is declared inside of `std::string`, it is sometimes written as `std::string::length()` in documentation.

We'll cover member functions, including how to write your own, in more detail later.

> **Key insight**
>
> With normal functions, we call `function(object)`. With member functions, we call `object.function()`.

Also note that `std::string::length()` returns an unsigned integral value (most likely of type `size_t`). If you want to assign the length to an `int` variable, you should `static_cast` it to avoid compiler warnings about signed/unsigned conversions:

```
int length { static_cast<int>(name.length()) };
```

> **For advanced readers**
>
> In C++20, you can also use the `std::ssize()` function to get the length of a `std::string` as a large signed integral type (usually `std::ptrdiff_t`):
>
> ```
> #include <iostream>
> #include <string>
>
> int main()
> {
> std::string name{ "Alex" };
> std::cout << name << " has " << std::ssize(name) << " characters\n";
>
> return 0;
> }
> ```
>
> Since a `ptrdiff_t` may be larger than an `int`, if you want to store the result of `std::ssize()` in an `int` variable, you should `static_cast` the result to an `int`:
>
> ```
> int len { static_cast<int>(std::ssize(name)) };
> ```

Initializing a `std::string` is expensive

Whenever a `std::string` is initialized, a copy of the string used to initialize it is made. Making copies of strings is expensive, so care should be taken to minimize the number of copies made.

Do not pass `std::string` by value

When a `std::string` is passed to a function by value, the `std::string` function parameter must be instantiated and initialized with the argument. This results in an expensive copy. We'll discuss what to do instead (use `std::string_view`) in lesson 5.8 – Introduction to std::string_view.

> **Best practice**
>
> Do not pass `std::string` by value, as it makes an expensive copy.

> **Tip**
>
> In most cases, use a `std::string_view` parameter instead (covered in lesson 5.8 – Introduction to std::string_view).

Returning a `std::string`

When a function returns by value to the caller, the return value is normally copied from the function back to the caller. So you might expect that you should not return `std::string` by value, as doing so would return an expensive copy of a `std::string`.

However, as a rule of thumb, it is okay to return a `std::string` by value when the expression of the return statement resolves to any of the following:

- A local variable of type `std::string`.
- A `std::string` that has been returned by value from another function call or operator.
- A `std::string` temporary that is created as part of the return statement.

> **For advanced readers**
>
> `std::string` supports a capability called move semantics, which allows an object that will be destroyed at the end of the function to instead be returned by value without making a copy. How move semantics works is beyond the scope of this introductory article, but is something we introduce in lesson 16.5 – Returning std::vector, and an introduction to move semantics.

In most other cases, prefer to avoid returning a `std::string` by value, as doing so will make an expensive copy.

> **Tip**
>
> If returning a C-style string literal, use a `std::string_view` return type instead (covered in lesson 5.9 – std::string_view (part 2)).

> **For advanced readers**
>
> In certain cases, `std::string` may also be returned by (const) reference, which is another way to avoid making a copy. We discuss this further in lessons 12.12 – Return by reference and return by address and 14.6 – Access functions.

Literals for `std::string`

Double-quoted string literals (like `"Hello, world!"`) are C-style strings by default (and thus, have a strange type).

We can create string literals with type `std::string` by using a `s` suffix after the double-quoted string literal. The `s` must be lower case.

```
#include <iostream>
#include <string> // for std::string

int main()
{
    using namespace std::string_literals; // easy access to the s suffix

    std::cout << "foo\n";   // no suffix is a C-style string literal
    std::cout << "goo\n"s;  // s suffix is a std::string literal

    return 0;
}
```

> **Tip**
>
> The “s”

5.8 — Introduction to std::string_view

Consider the following program:

```
#include <iostream>

int main()
{
    int x { 5 }; // x makes a copy of its initializer
    std::cout << x << '\n';

    return 0;
}
```

When the definition for `x` is executed, the initialization value `5` is copied into the memory allocated for variable `int x`. For fundamental types, initializing and copying a variable is fast.

Now consider this similar program:

```
#include <iostream>
#include <string>

int main()
{
    std::string s{ "Hello, world!" }; // s makes a copy of its initializer
    std::cout << s << '\n';

    return 0;
}
```

When `s` is initialized, the C-style string literal `"Hello, world!"` is copied into memory allocated for `std::string s`. Unlike fundamental types, initializing and copying a `std::string` is slow.

In the above program, all we do with `s` is print the value to the console, and then `s` is destroyed. We've essentially made a copy of "Hello, world!" just to print and then destroy that copy. That's inefficient.

We see something similar in this example:

```
#include <iostream>
#include <string>

void printString(std::string str) // str makes a copy of its initializer
{
    std::cout << str << '\n';
}

int main()
{
    std::string s{ "Hello, world!" }; // s makes a copy of its initializer
    printString(s);

    return 0;
}
```

This example makes two copies of the C-style string “Hello, world!”: one when we initialize `s` in `main()`, and another when we initialize parameter `str` in `printString()`. That’s a lot of needless copying just to print a string!

std::string_view C++17

To address the issue with `std::string` being expensive to initialize (or copy), C++17 introduced `std::string_view` (which lives in the `<string_view>` header). `std::string_view` provides read-only access to an *existing* string (a C-style string, a `std::string`, or another `std::string_view`) without making a copy. **Read-only** means that we can access and use the value being viewed, but we can not modify it.

The following example is identical to the prior one, except we’ve replaced `std::string` with `std::string_view`.

```
#include <iostream>
#include <string_view> // C++17

// str provides read-only access to whatever argument is passed in
void printSV(std::string_view str) // now a std::string_view
{
    std::cout << str << '\n';
}

int main()
{
    std::string_view s{ "Hello, world!" }; // now a std::string_view
    printSV(s);

    return 0;
}
```

This program produces the same output as the prior one, but no copies of the string “Hello, world!” are made.

When we initialize `std::string_view s` with C-style string literal `"Hello, world!"`, `s` provides read-only access to “Hello, world!” without making a copy of the string. When we pass `s` to `printSV()`, parameter `str` is initialized from `s`. This allows us to access “Hello, world!” through `str`, again without making a copy of the string.

> **Best practice**
>
> Prefer `std::string_view` over `std::string` when you need a read-only string, especially for function parameters.

`std::string_view` can be initialized with many different types of strings

One of the neat things about a `std::string_view` is how flexible it is. A `std::string_view` object can be initialized with a C-style string, a `std::string`, or another `std::string_view`:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    std::string_view s1 { "Hello, world!" }; // initialize with C-style string literal
    std::cout << s1 << '\n';

    std::string s{ "Hello, world!" };
    std::string_view s2 { s };  // initialize with std::string
    std::cout << s2 << '\n';

    std::string_view s3 { s2 }; // initialize with std::string_view
    std::cout << s3 << '\n';

    return 0;
}
```

`std::string_view` parameters will accept many different types of string arguments

Both a C-style string and a `std::string` will implicitly convert to a `std::string_view`. Therefore, a `std::string_view` parameter will accept arguments of type C-style string, a `std::string`, or `std::string_view`:

```
#include <iostream>
#include <string>
#include <string_view>

void printSV(std::string_view str)
{
    std::cout << str << '\n';
}

int main()
{
    printSV("Hello, world!"); // call with C-style string literal

    std::string s2{ "Hello, world!" };
    printSV(s2); // call with std::string

    std::string_view s3 { s2 };
    printSV(s3); // call with std::string_view

    return 0;
}
```

`std::string_view` will not implicitly convert to `std::string`

Because `std::string` makes a copy of its initializer (which is expensive), C++ won't allow implicit conversion of a `std::string_view` to a `std::string`. This is to prevent accidentally passing a `std::string_view` argument to a `std::string` parameter, and inadvertently making an expensive copy where such a copy may not be required.

However, if this is desired, we have two options:

1. Explicitly create a `std::string` with a `std::string_view` initializer (which is allowed, since this will rarely be done unintentionally)
2. Convert an existing `std::string_view` to a `std::string` using `static_cast`

The following example shows both options:

```
#include <iostream>
#include <string>
#include <string_view>

void printString(std::string str)
{
    std::cout << str << '\n';
}

int main()
{
    std::string_view sv{ "Hello, world!" };

    // printString(sv);   // compile error: won't implicitly convert std::string_view to a std::string

    std::string s{ sv }; // okay: we can create std::string using std::string_view initializer
    printString(s);      // and call the function with the std::string

    printString(static_cast<std::string>(sv)); // okay: we can explicitly cast a std::string_view to a std::s

    return 0;
}
```

Assignment changes what the `std::string_view` is viewing

Assigning a new string to a `std::string_view` causes the `std::string_view` to view the new string. It does not modify the prior string being viewed in any way.

The following example illustrates this:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    std::string name { "Alex" };
    std::string_view sv { name }; // sv is now viewing name
    std::cout << sv << '\n'; // prints Alex

    sv = "John"; // sv is now viewing "John" (does not change name)
    std::cout << sv << '\n'; // prints John

    std::cout << name << '\n'; // prints Alex

    return 0;
}
```

In the above example, `sv = "John"` causes `sv` to now view the string `"John"`. It does not change the value held by `name` (which is still `"Alex"`).

Literals for `std::string_view`

Double-quoted string literals are C-style string literals by default. We can create string literals with type `std::string_view` by using a `sv` suffix after the double-quoted string literal. The `sv` must be lower case.

```
#include <iostream>
#include <string>      // for std::string
#include <string_view> // for std::string_view

int main()
{
    using namespace std::string_literals;      // access the s suffix
    using namespace std::string_view_literals; // access the sv suffix

    std::cout << "foo\n";   // no suffix is a C-style string literal
    std::cout << "goo\n"s;  // s suffix is a std::string literal
    std::cout << "moo\n"sv; // sv suffix is a std::string_view literal

    return 0;
}
```

> **Related content**
>
> We discuss this use of `using namespace` in lesson 5.7 – Introduction to std::string. The same advice applies here.

It's fine to initialize a `std::string_view` object with a C-style string literal (you don't need to initialize it with a `std::string_view` literal).

That said, initializing a `std::string_view` using a `std::string_view` literal won't cause problems (as such literals are actually C-style string literals in disguise).

constexpr `std::string_view`

Unlike `std::string`, `std::string_view` has full support for constexpr:

```
#include <iostream>
#include <string_view>

int main()
{
    constexpr std::string_view s{ "Hello, world!" }; // s is a string symbolic constant
    std::cout << s << '\n'; // s will be replaced with "Hello, world!" at compile-time

    return 0;
}
```

This makes `constexpr std::string_view` the preferred choice when string symbolic constants are needed.

We will continue discussing `std::string_view` in the next lesson.

5.9 — std::string_view (part 2)

In prior lessons, we introduced two string types: `std::string` (5.7 – Introduction to std::string) and `std::string_view` (5.8 – Introduction to std::string_view).

Because `std::string_view` is our first encounter with a view type, we're going to spend some additional time discussing it further. We will focus on how to use `std::string_view` safely, and provide some examples illustrating how it can be used incorrectly. We'll conclude with some guidelines on when to use `std::string` vs `std::string_view`.

An introduction to owners and viewers

Let's sidebar into an analogy for a moment. Say you've decided that you're going to paint a picture of a bicycle. But you don't have a bicycle! What are you to do?

Well, you could go to the local cycle shop and buy one. You would own that bike. This has some benefits: you now have a bike that you can ride. You can guarantee the bike will always be available when you want it. You can decorate it, or move it. There are also some downsides to this choice. Bicycles are expensive. And if you buy one, you are now responsible for it. You have to periodically maintain it. And when you eventually decide you don't want it any more, you have to properly dispose of it.

Ownership can be expensive. As an owner, it is your responsibility to acquire, manage, and properly dispose of the objects you own.

On your way out of the house, you glance out your window front. You notice that your neighbor has parked their bike across from your window. You could just paint a picture of your neighbor's bike (as seen from your window) instead. There are lots of benefits to this choice. You save the expense of having to go acquire your own bike. You don't have to maintain it. Nor are you responsible for disposing of it. When you are done viewing, you can just shut your curtains and move on with your life. This ends your view of the object, but the object itself is not affected by this. There are also some potential downsides to this choice. You can't paint or customize your neighbors bike. And while you are viewing the bike, your neighbor may decide to change the way the bike looks, or move it out of your view altogether. You may end up with a view of something unexpected instead.

Viewing is inexpensive. As a viewer, you have no responsibility for the objects you are viewing, but you also have no control over those objects.

`std::string` is a (sole) owner

You might be wondering why `std::string` makes an expensive copy of its initializer. When an object is instantiated, memory is allocated for that object to store whatever data it needs to use throughout its lifetime. This memory is reserved for the object, and guaranteed to exist for as long as the object does. It is a safe space. `std::string` (and most other objects) copy the initialization value they are given into this memory so that they can have their own independent value to access and manipulate later. Once the initialization value has been copied, the object is no longer reliant on the initializer in any way.

And that's a good thing, because the initializer generally can't be trusted after initialization is complete. If you imagine the initialization process as a function call that initializes the object, who is passing in the initializer? The caller. When initialization is done, control returns back to the caller. At this point, the initialization statement is complete, and one of two things will typically happen:

- If the initializer was a temporary value or object, that temporary will be destroyed immediately.
- If the initializer was a variable, the caller still has access to that object. The caller can then do whatever it wants with the object, including modify or destroy it.

> **Key insight**
>
> An initialized object has no control over what happens to the initialization value after initialization is finished.

Because `std::string` makes its own copy of the initializer, it doesn't have to worry about what happens to the initializer after initialization is finished. The initializer can be destroyed, or modified, and it doesn't affect the `std::string`. The downside is that this independence comes with the cost of making an expensive copy.

In the context of our analogy, `std::string` is an owner – it is responsible for acquiring its string data from the initializer, managing access to the string data, and properly disposing of the string data when the `std::string` object is destroyed.

> **Key insight**
>
> In programming, when we call an object an owner, we generally mean that it is the sole owner (unless otherwise specified). Sole ownership (also

> called single ownership) ensures it is clear who has responsibility for that data.

We don't always need a copy

Let's revisit this example from the prior lesson:

```
#include <iostream>
#include <string>

void printString(std::string str) // str makes a copy of its initializer
{
    std::cout << str << '\n';
}

int main()
{
    std::string s{ "Hello, world!" };
    printString(s);

    return 0;
}
```

When `printString(s)` is called, `str` makes an expensive copy of `s`. The function prints the copied string and then destroys it.

Note that `s` is already holding the string we want to print. Could we just use the string that `s` is holding instead of making a copy? The answer is possibly – there are three criteria we need to assess:

- Could `s` be destroyed while `str` is still using it? No, `str` dies at the end of the function, and `s` exists in the scope of the caller and can't be destroyed before the function returns.
- Could `s` be modified while `str` is still using it? No, `str` dies at the end of the function, and the caller has no opportunity to modify the `s` before the function returns.
- Does `str` modify the string in some way that the caller would not expect? No, the function does not modify the string at all.

Since all three of these criteria are false, there is no risk in using the string that `s` is holding instead of making a copy. And since string copies are expensive, why pay for one that we don't need?

`std::string_view` is a viewer

`std::string_view` takes a different approach to initialization. Instead of making an expensive copy of the initialization string, `std::string_view` creates an inexpensive view *of* the initialization string. The `std::string_view` can then be used whenever access to the string is required.

In the context of our analogy, `std::string_view` is a viewer. It views an object that already exists elsewhere, and cannot modify that object. When the view is destroyed, the object being viewed is not affected. Having multiple viewers viewing an object simultaneously is fine.

It is important to note that a `std::string_view` remains dependent on the initializer through its lifetime. If the string being viewed is modified or destroyed while the view is still being used, unexpected or undefined behavior will result.

Whenever we use a view, it is up to us to ensure these possibilities do not occur.

> **Warning**
>
> A view is dependent on the object being viewed. If the object being viewed is modified or destroyed while the view is still being used, unexpected or undefined behavior will result.

A `std::string_view` that is viewing a string that has been destroyed is sometimes called a **dangling** view.

`std::string_view` is best used as a read-only function parameter

The best use for `std::string_view` is as a read-only function parameter. This allows us to pass in a C-style string, `std::string`, or `std::string_view` argument without making a copy, as the `std::string_view` will create a view to the argument.

```
#include <iostream>
#include <string>
#include <string_view>

void printSV(std::string_view str) // now a std::string_view, creates a view of the argument
{
    std::cout << str << '\n';
}

int main()
{
    printSV("Hello, world!"); // call with C-style string literal

    std::string s2{ "Hello, world!" };
    printSV(s2); // call with std::string

    std::string_view s3 { s2 };
    printSV(s3); // call with std::string_view

    return 0;
}
```

Because the `str` function parameter is created, initialized, used, and destroyed before control returns to the caller, there is no risk that the string being viewed (the function argument) will be modified or destroyed before our `str` parameter.

> **Should I prefer `std::string_view` or `const std::string&` function parameters? Advanced**
>
> Prefer `std::string_view` in most cases. We cover this topic further in lesson 12.6 – Pass by const lvalue reference.

Improperly using `std::string_view`

Let's take a look at a few cases where misusing `std::string_view` gets us into trouble.

Here's our first example:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    std::string_view sv{};

    { // create a nested block
        std::string s{ "Hello, world!" }; // create a std::string local to this nested block
        sv = s; // sv is now viewing s
    } // s is destroyed here, so sv is now viewing an invalid string

    std::cout << sv << '\n'; // undefined behavior

    return 0;
}
```

In this example, we're creating `std::string s` inside a nested block (don't worry about what a nested block is yet). Then we set `sv` to view `s`. `s` is then destroyed at the end of the nested block. `sv` doesn't know that `s` has been destroyed. When we then use `sv`, we are accessing an invalid object, and undefined behavior results.

Here's another variant of the same issue, where we initialize a `std::string_view` with the `std::string` return value of a function:

```
#include <iostream>
#include <string>
#include <string_view>

std::string getName()
{
    std::string s { "Alex" };
    return s;
}

int main()
{
  std::string_view name { getName() }; // name initialized with return value of function
  std::cout << name << '\n'; // undefined behavior

  return 0;
}
```

This behaves similarly to the prior example. The `getName()` function is returning a `std::string` containing the string "Alex". Return values are temporary objects that are destroyed at the end of the full expression containing the function call. We must either use this return value immediately, or copy it to use later.

But `std::string_view` doesn't make copies. Instead, it creates a view to the temporary return value, which is then destroyed. That leaves our `std::string_view` dangling (viewing an invalid object), and printing the view results in undefined behavior.

The following is a less-obvious variant of the above:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    using namespace std::string_literals;
    std::string_view name { "Alex"s }; // "Alex"s creates a temporary std::string
    std::cout << name << '\n'; // undefined behavior

    return 0;
}
```

A `std::string` literal (created via the `s` literal suffix) creates a temporary `std::string` object. So in this case, `"Alex"s` creates a temporary `std::string`, which we then use as the initializer for `name`. At this point, `name` is viewing the temporary `std::string`. Then the temporary `std::string` is destroyed, leaving `name` dangling. We get undefined behavior when we then use `name`.

> **Warning**
>
> Do not initialize a `std::string_view` with a `std::string` literal, as this will leave the `std::string_view` dangling.

It is okay to initialize a `std::string_view` with a C-style string literal or a `std::string_view` literal. It's also okay to initialize a `std::string_view` with a C-style string object, a `std::string` object, or a `std::string_view` object, as long as that string object outlives the view.

We can also get undefined behavior when the underlying string is modified:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    std::string s { "Hello, world!" };
    std::string_view sv { s }; // sv is now viewing s

    s = "Hello, a!";    // modifies s, which invalidates sv (s is still valid)
    std::cout << sv << '\n';   // undefined behavior

    return 0;
}
```

In this example, `sv` is again set to view `s`. `s` is then modified. When a `std::string` is modified, any views into that `std::string` are likely to be **invalidated**, meaning those views are now invalid or incorrect. Using an invalidated view will result in undefined behavior.

> **For advanced readers**
>
> - If the `std::string` reallocates memory in order to accommodate the new string data, it will return the memory used for the old string data back to the operating system. Since the `std::string_view` is still viewing the old string data, it is now dangling (pointing to a now-invalid object).
> - If the `std::string` does not reallocate memory, it will copy the new string data over the old string data (starting at the same memory address). The `std::string_view` will now be viewing the new string data (since it was placed at the same memory address as it was viewing), but it will not realize that the length of the `std::string` has probably changed. If the new string is longer than the old string, the `std::string_view` will now be viewing a substring of the new string (of the same length as the old string). If the new string is shorter than the old string, the `std::string_view` will now be viewing a superstring of the new string (consisting of the entire new string, plus whatever garbage characters are still in the memory beyond the end of the string).

> **Key insight**
>
> Modifying a `std::string` is likely to invalidate all views into that `std::string`.

Revalidating an invalid `std::string_view`

Invalidated objects can often be revalidated (made valid again) by setting them back to a known good state. For an invalidated `std::string_view`, we can do this by assigning the invalidated `std::string_view` object a valid string to view.

Here's the same example as prior, but we'll revalidate `sv`:

```
#include <iostream>
#include <string>
#include <string_view>

int main()
{
    std::string s { "Hello, world!" };
    std::string_view sv { s }; // sv is now viewing s

    s = "Hello, universe!";    // modifies s, which invalidates sv (s is still valid)
    std::cout << sv << '\n';   // undefined behavior

    sv = s;                    // revalidate sv: sv is now viewing s again
    std::cout << sv << '\n';   // prints "Hello, universe!"

    return 0;
}
```

After `sv` is invalidated by the modification of `s`, we revalidate `sv` via the statement `sv = s`, which causes `sv` to become a valid view of `s` again. When we print `sv` the second time, it prints "Hello, universe!".

Be careful returning a `std::string_view`

`std::string_view` can be used as the return value of a function. However, this is often dangerous.

Because local variables are destroyed at the end of the function, returning a `std::string_view` that is viewing a local variable will result in the returned `std::string_view` being invalid, and further use of that `std::string_view` will result in undefined behavior. For example:

```cpp
#include <iostream>
#include <string>
#include <string_view>

std::string_view getBoolName(bool b)
{
    std::string t { "true" };  // local variable
    std::string f { "false" }; // local variable

    if (b)
        return t;  // return a std::string_view viewing t

    return f; // return a std::string_view viewing f
} // t and f are destroyed at the end of the function

int main()
{
    std::cout << getBoolName(true) << ' ' << getBoolName(false) << '\n'; // undefined behavior

    return 0;
}
```

In the above example, when `getBoolName(true)` is called, the function returns a `std::string_view` that is viewing `t`. However, `t` is destroyed at the end of the function. This means the returned `std::string_view` is viewing an object that has been destroyed. So when the returned `std::string_view` is printed, undefined behavior results.

Your compiler may or may not warn you about such cases.

There are two main cases where

5.x — Chapter 5 summary and quiz

Chapter Review

A **constant** is a value that may not be changed during the program's execution. C++ supports two types of constants: named constants, and literals.

A **named constant** is a constant value that is associated with an identifier. A **Literal constant** is a constant value not associated with an identifier.

A variable whose value can not be changed is called a **constant variable**. The **const** keyword can be used to make a variable constant. Constant variables must be initialized. Avoid using `const` when passing by value or returning by value.

A **type qualifier** is a keyword that is applied to a type that modifies how that type behaves. As of C++23, C++ only supports `const` and `volatile` as type qualifiers.

A **constant expression** is an expression that can be evaluated at compile-time. An expression that is not a constant expression is sometimes called a **runtime expression**.

A **compile-time constant** is a constant whose value is known at compile-time. A **runtime constant** is a constant whose initialization value isn't known until runtime.

A **constexpr** variable must be a compile-time constant, and initialized with a constant expression. Function parameters cannot be constexpr.

Literals are values inserted directly into the code. Literals have types, and literal suffixes can be used to change the type of a literal from the default type.

A **magic number** is a literal (usually a number) that either has an unclear meaning or may need to be changed later. Don't use magic numbers in your code. Instead, use symbolic constants.

In everyday life, we count using **decimal** numbers, which have 10 digits. Computers use **binary**, which only has 2 digits. C++ also supports **octal** (base 8) and **hexadecimal** (base 16). These are all examples of **numeral systems**, which are collections of symbols (digits) used to represent numbers.

A **string** is a collection of sequential characters that is used to represent text (such as names, words, and sentences). String literals are always placed between double quotes. String literals in C++ are C-style strings, which have a strange type that is hard to work with.

std::string offers an easy and safe way to deal with text strings. std::string lives in the <string> header. `std::string` is expensive to initialize (or assign to) and copy.

std::string_view provides read-only access to an existing string (a C-style string literal, a std::string, or a char array) without making a copy. A `std::string_view` that is viewing a string that has been destroyed is sometimes called a **dangling** view. When a `std::string` is modified, all views into that `std::string` are **invalidated**, meaning those views are now invalid. Using an invalidated view (other than to revalidate it) will produce undefined behavior.

Because C-style string literals exist for the entire program, it is okay to set a `std::string_view` to a C-style string literal, and even return such a `std::string_view` from a function.

A **substring** is a contiguous sequence of characters within an existing string.

Quiz time

Why are named constants often a better choice than literal constants?

> **Solution**
>
> Using literal constants (a.k.a. magic numbers) in your program makes your program harder to understand and harder to modify. Symbolic constants help document what the numbers actually represent, and changing a symbolic constant at its declaration changes the value everywhere it is used.

Why are const/constexpr variables usually a better choice than #defined symbolic constants?

> **Solution**
>
> `#define` constants do not show up in the debugger and are more likely to have naming conflicts.

Find 3 issues in the following code:

```cpp
#include <cstdint> // for std::uint8_t
#include <iostream>

int main()
{
  std::cout << "How old are you?\n";

  std::uint8_t age{};
  std::cin >> age;

  std::cout << "Allowed to drive a car in Texas: ";

  if (age >= 16)
      std::cout << "Yes";
  else
      std::cout << "No";

  std::cout << '.\n';

  return 0;
}
```

Sample desired output:

```
How old are you?
6
Allowed to drive a car in Texas: No
```

```
How old are you?
19
Allowed to drive a car in Texas: Yes
```

Solution

1. On line 8, `age` is defined as a `std::uint8_t`. Because `std::uint8_t` is typically defined as a char type, using it here will cause the program to behave as if we're inputting and outputting a char value rather than a numeric value. For example, if the user entered their age as "18", only the character `'1'` would be extracted. Because `1` has ASCII value `49`, the user would be treated as if they were 49 years old. A regular `int` should be used to store the age, as the age doesn't require a specific minimum integer width. We can also remove `#include <cstdint>`.
2. On line 13, we use the magic number `16`. Although the meaning of `16` is clear from the context it is used in, using a `constexpr` variable with the value `16` should be preferred instead.
3. On line 18, `'.\n'` is a multicharacter literal that will print the wrong value. It should be double-quoted (`".\n"`).

What are the primary differences between `std::string` and `std::string_view`?

What can go wrong when using a `std::string_view`?

Solution

`std::string` provides a modifiable string. It is expensive to initialize and copy.

`std::string_view` provide a read-only view of a string that exists elsewhere. It is inexpensive to initialize and copy. `std::string_view` can be dangerous when the string being viewed is destroyed before the `std::string_view` that is viewing it.

Write a program that asks for the name and age of two people, then prints which person is older.

Here is the sample output from one run of the program:

```
Enter the name of person #1: John Bacon
Enter the age of John Bacon: 37
Enter the name of person #2: David Jenkins
Enter the age of David Jenkins: 44
David Jenkins (age 44) is older than John Bacon (age 37).
```

Hint: Input the person's name using `std::getline()`

Solution

```cpp
#include <iostream>
#include <string>
#include <string_view>

std::string getName(int num)
{
    std::cout << "Enter the name of person #" << num << ": ";
    std::string name{};
    std::getline(std::cin >> std::ws, name); // read a full line of text into name

    return name;
}

int getAge(std::string_view sv)
{
    std::cout << "Enter the age of " << sv << ": ";
    int age{};
    std::cin >> age;

    return age;
}

void printOlder(std::string_view name1, int age1, std::string_view name2, int age2)
{
    if (age1 > age2)
        std::cout << name1 << " (age " << age1 <<") is older than " << name2 << " (age " << age2 <<").\n";
    else
        std::cout << name2 << " (age " << age2 <<") is older than " << name1 << " (age " << age1 <<").\n";
}

int main()
{
    const std::string name1{ getName(1) };
    const int age1 { getAge(name1) };

    const std::string name2{ getName(2) };
    const int age2 { getAge(name2) };

    printOlder(name1, age1, name2, age2);

    return 0;
}
```

In the solution to the above quiz, why can't variable `age1` in `main` be constexpr?

Solution

A constexpr variable requires a constant expression initializer, and the call to `getAge()` isn't allowed in a constant expression. Therefore, we can only make the variable const.